

T1 – Trabalho de Sistemas Distribuídos

Professor Cesar Augusto de Rose – Turma 030

Grupo: Arthur Greco, Cristiano José Ferrazzo e Germano Bruscato Corrêa

Introdução

Em sistemas distribuídos, muitas vezes é necessário eleger um líder. A eleição de líderes é um processo essencial para garantir a continuidade do serviço diante de falhas ou mudanças na configuração dos processos participantes. Dentro do sistema, cada processo conhece o anel inteiro, mas manda mensagens somente para o próximo processo ativo. Por exemplo, sistemas como bancos de dados distribuídos frequentemente elegem um líder para coordenar transações e garantir consistência dos dados. Um sistema de pagamentos do sistema financeiro, que possui um único servidor para validar a transação em tempo real, precisa eleger um líder (novo servidor de validação), caso o servidor principal fique indisponível. Isso pode ser feito por meio de uma eleição.

Descrição do algoritmo

O algoritmo de eleição implementado neste código é o algoritmo de eleição em anel (ring), no qual os processos estão organizados em um anel e se comunicam apenas com seu próximo vizinho. Cada processo conhece o anel e o líder atual. Quando um processo detecta ou é instruído a iniciar uma eleição (por exemplo, após a falha do líder), ele envia uma mensagem circulando pelo anel uma mensagem onde os processos depositam suas votações. Em nossa implementação, a mensagem contém o identificador de maior prioridade de processo observado até o momento. Nessa implementação, cada processo compara seu próprio ID com o da mensagem e, se for maior, substitui o valor no corpo da mensagem. Quando a mensagem retorna ao processo que disparou a eleição, o processo com maior prioridade é considerado o novo líder, e então, e uma

mensagem de confirmação percorre o anel informando todos os processos.

Tipos de Mensagens Implementadas

As seguintes mensagens foram implementadas no código elaborado:

1. Forçar falha: Define como verdadeira uma flag interna que indica uma falha simulada. Neste estado, o processo ignora mensagens normais, repassando-as adiante sem processamento.
2. Forçar retorno: Define como falsa a flag de falha interna, permitindo que o processo volte a processar mensagens normalmente.
3. Votar na eleição: Quando recebe esta mensagem, o processo:
 - a) Verifica se ainda não votou. Se não tiver votado, compara seu *TaskId* com o recebido na mensagem. Caso possua maior prioridade, substitui o *TaskId* da mensagem pelo próprio; caso contrário, mantém o recebido. Em seguida, envia a mensagem ao próximo processo e marca internamente que já votou.
 - b) Caso já tenha votado, percebe que a eleição deu uma volta completa no anel e envia uma mensagem de confirmação ao próximo processo, indicando como líder o *TaskId* contido na mensagem.
4. Confirmação da eleição: Ao receber esta mensagem, o processo limpa sua flag interna de votação.
 - a) Se ainda não confirmou a eleição atual, armazena o líder indicado na mensagem, repassa a mensagem ao próximo processo e ajusta sua flag

interna para indicar que já confirmou.

b) Caso já tenha confirmado, envia uma mensagem ao canal de controle, indicando o encerramento da eleição.

5. Finalização do processo: Interrompe o loop interno que aguarda por mensagens.

Modelagem das Falhas

As falhas são simuladas por uma flag interna. Quando ativada, o processo repassa diretamente as mensagens recebidas sem qualquer processamento, exceto a mensagem específica para sair da falha. O processo de controle é responsável por ativar ou desativar essa condição de falha simulada.

Testes e Papel do Processo Controle

Como o código não está rodando em um sistema distribuído verdadeiro, o processo controle teve o papel de introduzir as falhas artificialmente no anel, simulando a indisponibilidade e a reativação dos processos.

Foram realizados testes específicos:

1. Remoção e retorno do líder atual seguido por disparo de novas eleições, validando a correta definição do líder.
2. Remoção de um processo não líder, confirmando que a eleição subsequente não altera o líder.
3. Remoção de todos os processos exceto um, disparando a eleição e depois retornando os processos removidos, verificando que a eleição subsequente ocorre sem problemas nos processos que haviam sido removidos.

Detalhes da implementação do algoritmo

1. As ações iniciadas pelo processo de controle notificam o canal de controle ao

serem concluídas.

2. Mudanças no estado de falha notificam o controle imediatamente; ações relacionadas à eleição aguardam o término da eleição e confirmação para notificar.
3. Ao iniciar uma eleição, o processo recebe uma mensagem contendo o *TaskId* do candidato mais prioritário até então. O processo compara seu *TaskId* com o recebido, atualizando-o caso possua prioridade maior e então repassa a mensagem. Na nossa implementação, a prioridade de um processo sob outro se dá pela numeração de identificação do processo - quanto maior, mais prioritário
4. A confirmação de eleição ocorre quando uma mensagem de votação retorna a um processo que já votou, indicando que a eleição percorreu todo o anel. O mesmo ocorre para a confirmação, finalizando a rodada.

Recursos adicionais implementados

1. Enumerações para organizar códigos dos tipos de mensagens.
2. Corpo das mensagens simplificado, contendo apenas o *TaskId* mais prioritário da eleição.
3. Manutenção do wait group para sinalizar ao processo principal quando subprocessos são encerrados.

Nos processos do anel, além das estruturas originais, foram adicionadas:

1. Uma flag para indicar se já houve voto na eleição atual, resetada apenas após a confirmação.
2. Uma flag para indicar se o processo já confirmou a eleição atual, resetada apenas quando ocorre uma nova eleição.
3. Um loop contínuo que garante ao processo a espera constante por novas mensagens, até receber uma mensagem de finalização.

```
$ go run election.go
```

```
Anel de processos criado
```

```
Processo controlador criado
```

```
===== Teste 1: derrubar e retornar
líder, eleição deve escolher novo enquanto está
fora e voltar para o líder original depois
```

```
Controle: mudar o processo 3 para falho
```

```
3: aguardando mensagens
```

```
0: aguardando mensagens
```

```
3: mensagem recebida!
```

```
3: recebi mensagem do tipo 2, conteúdo: [ 0 ]
```

```
3: falhei
```

```
3: líder atual 0
```

```
3: aguardando mensagens
```

```
Controle: confirmação do processo 3
```

```
Controle: dispara eleição no processo 0
```

```
0: mensagem recebida!
```

```
0: recebi mensagem do tipo 4, conteúdo: [ -1 ]
```

```
0: tenho prioridade em relação a -1, me
```

```
colocando na eleição
```

```
2: aguardando mensagens
```

```
1: aguardando mensagens
```

```
1: mensagem recebida!
```

```
1: recebi mensagem do tipo 4, conteúdo: [ 0 ]
```

```
1: tenho prioridade em relação a 0, me
```

```
colocando na eleição
```

```
1: aguardando mensagens
```

```
2: mensagem recebida!
```

```
2: recebi mensagem do tipo 4, conteúdo: [ 1 ]
```

```
2: tenho prioridade em relação a 1, me
```

```
colocando na eleição
```

```
2: aguardando mensagens
```

```
3: mensagem recebida!
```

```
3: recebi mensagem do tipo 4, conteúdo: [ 2 ]
```

```
3: estou fora, pulando para o próximo
```

```
0: aguardando mensagens
```

```
0: mensagem recebida!
```

```
0: recebi mensagem do tipo 4, conteúdo: [ 2 ]
```

```
0: eu disparei a eleição, enviando confirmação
```

```
do vencedor: 2
```

```
0: aguardando mensagens
```

```
1: mensagem recebida!
```

```
1: recebi mensagem do tipo 5, conteúdo: [ 2 ]
```

```
1: confirmando resultado da eleição, processo
```

```
líder é 2
```

```
1: aguardando mensagens
```

```
2: mensagem recebida!
```

```
2: recebi mensagem do tipo 5, conteúdo: [ 2 ]
```

```
2: confirmando resultado da eleição, processo
```

```
líder é 2
```

```
3: aguardando mensagens
```

```
3: mensagem recebida!
```

```
3: recebi mensagem do tipo 5, conteúdo: [ 2 ]
```

```
3: estou fora, pulando para o próximo
```

```
3: aguardando mensagens
```

```
0: mensagem recebida!
```

```
0: recebi mensagem do tipo 5, conteúdo: [ 2 ]
```

```
0: confirmando resultado da eleição, processo
```

```
líder é 2
```

```
0: aguardando mensagens
```

```
1: mensagem recebida!
```

```
1: recebi mensagem do tipo 5, conteúdo: [ 2 ]
```

```
1: aguardando mensagens
```

```
Controle: confirmação da eleição vinda do
processo 1
```

```
Controle: retorna processo 3
```

```
3: mensagem recebida!
```

```
3: recebi mensagem do tipo 3, conteúdo: [ 0 ]
```

```
3: voltei da falha
```

```
3: líder atual 0
```

```
3: aguardando mensagens
```

```
Controle: confirmação do processo 3
```

```
Controle: dispara eleição no processo 3
```

```
3: mensagem recebida!
```

```
3: recebi mensagem do tipo 4, conteúdo: [ -1 ]
```

```
3: tenho prioridade em relação a -1, me
colocando na eleição
```

```
3: aguardando mensagens
```

```
0: mensagem recebida!
```

```
0: recebi mensagem do tipo 4, conteúdo: [ 3 ]
```

```
0: TaskId na eleição com mais prioridade que
```

```
eu
```

```
0: aguardando mensagens
```

```
1: mensagem recebida!
```

```
1: recebi mensagem do tipo 4, conteúdo: [ 3 ]
```

```
1: TaskId na eleição com mais prioridade que
```

```
eu
```

```
2: aguardando mensagens
```

```
2: mensagem recebida!
```

```
2: recebi mensagem do tipo 4, conteúdo: [ 3 ]
```

```
2: TaskId na eleição com mais prioridade que
```

```
eu
```

```
2: aguardando mensagens
```

```
3: mensagem recebida!
```

```
3: recebi mensagem do tipo 4, conteúdo: [ 3 ]
```

```
3: eu disparei a eleição, enviando confirmação
do vencedor: 3
```

```
3: aguardando mensagens
```

```
0: mensagem recebida!
```

```
0: recebi mensagem do tipo 5, conteúdo: [ 3 ]
```

```
0: confirmando resultado da eleição, processo
```

```
líder é 3
```

```
1: aguardando mensagens
```

```
1: mensagem recebida!
```

```
1: recebi mensagem do tipo 5, conteúdo: [ 3 ]
```

```
1: confirmando resultado da eleição, processo
```

```
líder é 3
```

```
0: aguardando mensagens
```

```
1: aguardando mensagens
```

```
2: mensagem recebida!
```

```
2: recebi mensagem do tipo 5, conteúdo: [ 3 ]
```

```
2: confirmando resultado da eleição, processo
```

```
líder é 3
```

```
2: aguardando mensagens
```

```
3: mensagem recebida!
```

```
3: recebi mensagem do tipo 5, conteúdo: [ 3 ]
```

```
3: confirmando resultado da eleição, processo
```

```
líder é 3
```

```
3: aguardando mensagens
```

```
0: mensagem recebida!
```

```
0: recebi mensagem do tipo 5, conteúdo: [ 3 ]
```

```
0: aguardando mensagens
```

```
Controle: confirmação da eleição vinda do
processo 0
```

===== Teste 2: derruba processo menor,
eleição deve manter mesmo líder

Controle: mudar o processo 0 para falho

0: mensagem recebida!
0: recebi mensagem do tipo 2, conteúdo: [0]
0: falhei
0: líder atual 3
0: aguardando mensagens

Controle: confirmação do processo 0

Controle: dispara eleição no processo 1

1: mensagem recebida!
1: recebi mensagem do tipo 4, conteúdo: [-1]
1: tenho prioridade em relação a -1, me colocando na eleição
1: aguardando mensagens
2: mensagem recebida!
2: recebi mensagem do tipo 4, conteúdo: [1]
2: tenho prioridade em relação a 1, me colocando na eleição
2: aguardando mensagens
3: mensagem recebida!
3: recebi mensagem do tipo 4, conteúdo: [2]
3: tenho prioridade em relação a 2, me colocando na eleição
3: aguardando mensagens
0: mensagem recebida!
0: recebi mensagem do tipo 4, conteúdo: [3]
0: estou fora, pulando para o próximo
0: aguardando mensagens
1: mensagem recebida!
1: recebi mensagem do tipo 4, conteúdo: [3]
1: eu disparei a eleição, enviando confirmação do vencedor: 3
1: aguardando mensagens
2: mensagem recebida!
2: recebi mensagem do tipo 5, conteúdo: [3]
2: confirmando resultado da eleição, processo líder é 3
2: aguardando mensagens
3: mensagem recebida!
3: recebi mensagem do tipo 5, conteúdo: [3]
3: confirmando resultado da eleição, processo líder é 3
3: aguardando mensagens
0: mensagem recebida!
0: recebi mensagem do tipo 5, conteúdo: [3]
0: estou fora, pulando para o próximo
0: aguardando mensagens
1: mensagem recebida!
1: recebi mensagem do tipo 5, conteúdo: [3]
1: confirmando resultado da eleição, processo líder é 3
1: aguardando mensagens
2: mensagem recebida!
2: recebi mensagem do tipo 5, conteúdo: [3]
2: aguardando mensagens

Controle: confirmação da eleição vinda do processo 2

Controle: retorna processo 0

0: mensagem recebida!
0: recebi mensagem do tipo 3, conteúdo: [0]
0: voltei da falha
0: líder atual 3
0: aguardando mensagens

Controle: confirmação do processo 0

Controle: dispara eleição no processo 1

1: mensagem recebida!
1: recebi mensagem do tipo 4, conteúdo: [-1]
1: tenho prioridade em relação a -1, me colocando na eleição
1: aguardando mensagens
2: mensagem recebida!
2: recebi mensagem do tipo 4, conteúdo: [1]
2: tenho prioridade em relação a 1, me colocando na eleição
2: aguardando mensagens
3: mensagem recebida!
3: recebi mensagem do tipo 4, conteúdo: [2]
3: tenho prioridade em relação a 2, me colocando na eleição
3: aguardando mensagens
0: mensagem recebida!
0: recebi mensagem do tipo 4, conteúdo: [3]
0: TaskId na eleição com mais prioridade que eu
0: aguardando mensagens
1: mensagem recebida!
1: recebi mensagem do tipo 4, conteúdo: [3]
1: eu disparei a eleição, enviando confirmação do vencedor: 3
1: aguardando mensagens
2: mensagem recebida!
2: recebi mensagem do tipo 5, conteúdo: [3]
2: confirmando resultado da eleição, processo líder é 3
2: aguardando mensagens
3: mensagem recebida!
3: recebi mensagem do tipo 5, conteúdo: [3]
3: confirmando resultado da eleição, processo líder é 3
3: aguardando mensagens
0: mensagem recebida!
0: recebi mensagem do tipo 5, conteúdo: [3]
0: confirmando resultado da eleição, processo líder é 3
0: aguardando mensagens
1: mensagem recebida!
1: recebi mensagem do tipo 5, conteúdo: [3]
1: confirmando resultado da eleição, processo líder é 3
1: aguardando mensagens
2: mensagem recebida!
2: recebi mensagem do tipo 5, conteúdo: [3]
2: aguardando mensagens

Controle: confirmação da eleição vinda do processo 2

===== Teste 3: derruba todos processos
menos um

Controle: mudar o processo 0 para falho

0: mensagem recebida!
0: recebi mensagem do tipo 2, conteúdo: [0]
0: falhei
0: lider atual 3
0: aguardando mensagens

Controle: confirmação do processo 0

Controle: mudar o processo 1 para falho

1: mensagem recebida!
1: recebi mensagem do tipo 2, conteúdo: [0]
1: falhei
1: lider atual 3
1: aguardando mensagens

Controle: confirmação do processo 1

Controle: mudar o processo 3 para falho

3: mensagem recebida!
3: recebi mensagem do tipo 2, conteúdo: [0]
3: falhei
3: lider atual 3
3: aguardando mensagens

Controle: confirmação do processo 3

Controle: dispara eleição no processo 2

2: mensagem recebida!
2: recebi mensagem do tipo 4, conteúdo: [-1]
2: tenho prioridade em relação a -1, me colocando na eleição
2: aguardando mensagens
3: mensagem recebida!
3: recebi mensagem do tipo 4, conteúdo: [2]
3: estou fora, pulando para o próximo
3: aguardando mensagens
0: mensagem recebida!
0: recebi mensagem do tipo 4, conteúdo: [2]
0: estou fora, pulando para o próximo
0: aguardando mensagens
1: mensagem recebida!
1: recebi mensagem do tipo 4, conteúdo: [2]
1: estou fora, pulando para o próximo
1: aguardando mensagens
2: mensagem recebida!
2: recebi mensagem do tipo 4, conteúdo: [2]
2: eu disparei a eleição, enviando confirmação do vencedor: 2

2: aguardando mensagens
3: mensagem recebida!
3: recebi mensagem do tipo 5, conteúdo: [2]
3: estou fora, pulando para o próximo
3: aguardando mensagens
0: mensagem recebida!
0: recebi mensagem do tipo 5, conteúdo: [2]
0: estou fora, pulando para o próximo
0: aguardando mensagens
1: mensagem recebida!
1: recebi mensagem do tipo 5, conteúdo: [2]
1: estou fora, pulando para o próximo
1: aguardando mensagens
2: mensagem recebida!
2: recebi mensagem do tipo 5, conteúdo: [2]
2: confirmando resultado da eleição, processo líder é 2
2: aguardando mensagens
3: mensagem recebida!
3: recebi mensagem do tipo 5, conteúdo: [2]

3: estou fora, pulando para o próximo
3: aguardando mensagens
0: mensagem recebida!
0: recebi mensagem do tipo 5, conteúdo: [2]
0: estou fora, pulando para o próximo
0: aguardando mensagens
1: mensagem recebida!
1: recebi mensagem do tipo 5, conteúdo: [2]
1: estou fora, pulando para o próximo
1: aguardando mensagens
2: mensagem recebida!
2: recebi mensagem do tipo 5, conteúdo: [2]
2: aguardando mensagens

Controle: confirmação da eleição vinda do processo 2

Controle: retorna processo 0

0: mensagem recebida!
0: recebi mensagem do tipo 3, conteúdo: [0]
0: voltei da falha
0: lider atual 3
0: aguardando mensagens

Controle: confirmação do processo 0

Controle: retorna processo 1

1: mensagem recebida!
1: recebi mensagem do tipo 3, conteúdo: [0]
1: voltei da falha
1: lider atual 3
1: aguardando mensagens

Controle: confirmação do processo 1

Controle: retorna processo 3

3: mensagem recebida!
3: recebi mensagem do tipo 3, conteúdo: [0]
3: voltei da falha
3: lider atual 3
3: aguardando mensagens

Controle: confirmação do processo 3

Controle: dispara eleição no processo 0

0: mensagem recebida!
0: recebi mensagem do tipo 4, conteúdo: [-1]
0: tenho prioridade em relação a -1, me colocando na eleição
0: aguardando mensagens
1: mensagem recebida!
1: recebi mensagem do tipo 4, conteúdo: [0]
1: tenho prioridade em relação a 0, me colocando na eleição

1: aguardando mensagens
2: mensagem recebida!
2: recebi mensagem do tipo 4, conteúdo: [1]
2: tenho prioridade em relação a 1, me colocando na eleição
2: aguardando mensagens
3: mensagem recebida!
3: recebi mensagem do tipo 4, conteúdo: [2]
3: tenho prioridade em relação a 2, me colocando na eleição

3: aguardando mensagens
0: mensagem recebida!
0: recebi mensagem do tipo 4, conteúdo: [3]
0: eu disparei a eleição, enviando confirmação do vencedor: 3

0: aguardando mensagens
1: mensagem recebida!
1: recebi mensagem do tipo 5, conteúdo: [3]
1: confirmando resultado da eleição, processo líder é 3
1: aguardando mensagens
2: mensagem recebida!

```

2: recebi mensagem do tipo 5, conteúdo: [ 3 ]
2: confirmando resultado da eleição, processo
líder é 3
2: aguardando mensagens
3: mensagem recebida!
3: recebi mensagem do tipo 5, conteúdo: [ 3 ]
3: confirmando resultado da eleição, processo
líder é 3
3: aguardando mensagens
0: mensagem recebida!
0: recebi mensagem do tipo 5, conteúdo: [ 3 ]
0: confirmando resultado da eleição, processo
líder é 3
0: aguardando mensagens
1: mensagem recebida!
1: recebi mensagem do tipo 5, conteúdo: [ 3 ]
1: aguardando mensagens
Controle: confirmação da eleição vinda do
processo 1
Disparando finalizações
    Processo controlador concluído

0: mensagem recebida!

```

```

0: recebi mensagem do tipo 6, conteúdo: [ 0 ]
0: finalizando o processo
0: terminei
1: mensagem recebida!
1: recebi mensagem do tipo 6, conteúdo: [ 0 ]
1: finalizando o processo
1: terminei
2: mensagem recebida!
2: recebi mensagem do tipo 6, conteúdo: [ 0 ]
2: finalizando o processo
2: terminei
3: mensagem recebida!
3: recebi mensagem do tipo 6, conteúdo: [ 0 ]
3: finalizando o processo
3: terminei

```

```

001. package main
002.
003. import (
004.     "fmt"
005.     "sync"
006. )
007.
008. type mensagem struct {
009.     tipo int // tipo da mensagem para fazer o controle do que fazer (eleição, confirmacao da eleicao)
010.     corpo int // conteudo da mensagem para colocar os ids (usar um tamanho compativel com o numero de
    processos no anel)
011. }
012.
013. const (
014.     TIPO_MSG_FORCE_FALHA int = 2
015.     TIPO_MSG_FORCE_RETURN int = 3
016.     TIPO_MSG_VOTE_ELECTION int = 4
017.     TIPO_MSG_CONFIRM_ELECTION int = 5
018.     TIPO_MSG_FINISH_PROCESS int = 6
019. )
020.
021. var (
022.     chans = []chan mensagem{ // vetor de canias para formar o anel de eleicao - chan[0], chan[1] and chan[2] ...
023.         make(chan mensagem),
024.         make(chan mensagem),
025.         make(chan mensagem),
026.         make(chan mensagem),
027.     }
028.     controle = make(chan int)
029.     firstLeader = 3
030.     highestProcess = 3
031.     wg sync.WaitGroup // wg is used to wait for the program to finish
032. )
033.
034. func normalizeProcessId(TaskId int) int {
035.     if TaskId > highestProcess {
036.         return 0
037.     } else if TaskId < 0 {
038.         return highestProcess
039.     }
040.
041.     return TaskId
042. }
043.
044. func getInputChannelForTaskId(TaskId int) int {
045.     return normalizeProcessId(TaskId - 1)
046. }
047.
048. func derrubarProcesso(controle chan int, TaskId int) {
049.     fmt.Printf("Controle: mudar o processo %d para falho\n", TaskId)
050.     inputChannel := getInputChannelForTaskId(TaskId)
051.     chans[inputChannel] <- mensagem{
052.         tipo: TIPO_MSG_FORCE_FALHA,
053.     }
054.     fmt.Printf("Controle: confirmação do processo %d\n", <-controle)
055. }
056.

```

```

057. func voltaProcesso(controle chan int, TaskId int) {
058.     fmt.Printf("Controle: retorna processo %d\n", TaskId)
059.     inputChannel := getInputChannelForTaskId(TaskId)
060.     chans[inputChannel] <- mensagem{
061.         tipo: TIPO_MSG_FORCE_RETURN,
062.     }
063.     fmt.Printf("Controle: confirmação do processo %d\n", <-controle)
064. }
065.
066. func disparaEleicao(controle chan int, TaskId int) {
067.     fmt.Printf("Controle: dispara eleição no processo %d\n", TaskId)
068.     inputChannel := getInputChannelForTaskId(TaskId)
069.     chans[inputChannel] <- mensagem{
070.         tipo: TIPO_MSG_VOTE_ELECTION,
071.         corpo: -1,
072.     }
073.     fmt.Printf("Controle: confirmação da eleição vinda do processo %d\n", <-controle)
074. }
075.
076. func finalizaProcessos() {
077.     fmt.Printf("Disparando finalizações")
078.
079.     chans[0] <- mensagem{tipo: TIPO_MSG_FINISH_PROCESS}
080.     chans[1] <- mensagem{tipo: TIPO_MSG_FINISH_PROCESS}
081.     chans[2] <- mensagem{tipo: TIPO_MSG_FINISH_PROCESS}
082.     chans[3] <- mensagem{tipo: TIPO_MSG_FINISH_PROCESS}
083. }
084.
085. func ElectionControler(in chan int) {
086.     defer wg.Done()
087.
088.     fmt.Println("\n\n\n===== Teste 1: derrubar e retornar líder, eleição deve escolher novo enquanto
está fora e voltar para o líder original depois\n")
089.     derrubarProcesso(in, firstLeader)
090.     disparaEleicao(in, normalizeProcessId(firstLeader+1))
091.     voltaProcesso(in, firstLeader)
092.     disparaEleicao(in, firstLeader)
093.
094.     fmt.Println("\n\n\n===== Teste 2: derruba processo menor, eleição deve manter mesmo líder\n")
095.     derrubarProcesso(in, 0)
096.     disparaEleicao(in, 1)
097.     voltaProcesso(in, 0)
098.     disparaEleicao(in, 1)
099.
100.     fmt.Println("\n\n\n===== Teste 3: derruba todos processos menos um\n")
101.     derrubarProcesso(in, 0)
102.     derrubarProcesso(in, 1)
103.     derrubarProcesso(in, 3)
104.     disparaEleicao(in, 2)
105.     voltaProcesso(in, 0)
106.     voltaProcesso(in, 1)
107.     voltaProcesso(in, 3)
108.     disparaEleicao(in, 0)
109.
110.     finalizaProcessos()
111.     fmt.Println("\n Processo controlador concluído\n")
112. }

```



```

113.
114.     func ElectionStage(TaskId int, in chan mensagem, out chan mensagem, leader int, colorCode string) {
115.         defer wg.Done()
116.
117.         var actualLeader int
118.         var bFailed = false
119.         var alreadyVoted = false
120.         var alreadyConfirmed = false
121.
122.         actualLeader = leader
123.
124.         var keepListening = true
125.         for keepListening {
126.             fmt.Printf(colorCode+"%2d: aguardando mensagens\n\033[0m", TaskId)
127.             inboundMessage := <-in
128.             fmt.Printf(colorCode+"%2d: mensagem recebida!\n\033[0m", TaskId)
129.
130.             fmt.Printf(colorCode+"%2d: recebi mensagem do tipo %d, conteúdo: [ %d ]\n\033[0m", TaskId, inbound-
Message.tipo, inboundMessage.corpo)
131.
132.             if bFailed && inboundMessage.tipo != TIPO_MSG_FORCE_RETURN {
133.                 fmt.Printf(colorCode+"%2d: estou fora, pulando para o próximo\n\033[0m", TaskId)
134.                 out <- inboundMessage
135.                 continue
136.             }
137.
138.             switch inboundMessage.tipo {
139.             case TIPO_MSG_FORCE_FALHA:
140.                 {
141.                     bFailed = true
142.                     fmt.Printf(colorCode+"%2d: falhei \n\033[0m", TaskId)
143.                     fmt.Printf(colorCode+"%2d: lider atual %d\n\033[0m", TaskId, actualLeader)
144.                     controle <- TaskId
145.                 }
146.             case TIPO_MSG_FORCE_RETURN:
147.                 {
148.                     bFailed = false
149.                     fmt.Printf(colorCode+"%2d: voltei da falha \n\033[0m", TaskId)
150.                     fmt.Printf(colorCode+"%2d: lider atual %d\n\033[0m", TaskId, actualLeader)
151.                     controle <- TaskId
152.                 }
153.             case TIPO_MSG_VOTE_ELECTION:
154.                 {
155.                     alreadyConfirmed = false
156.                     if alreadyVoted {
157.                         alreadyVoted = false
158.                         out <- mensagem{
159.                             tipo: TIPO_MSG_CONFIRM_ELECTION,
160.                             corpo: inboundMessage.corpo,
161.                         }
162.                         fmt.Printf(colorCode+"%2d: eu disparei a eleição, enviando confirmação do vencedor:
163.                             %2d\n\033[0m", TaskId, inboundMessage.corpo)
164.                     } else {
165.                         alreadyVoted = true
166.                         if inboundMessage.corpo < TaskId {
167.                             fmt.Printf(colorCode+"%2d: tenho prioridade em relação a %2d, me colocando na elei-
ção\n\033[0m", TaskId, inboundMessage.corpo)

```

```

167.         inboundMessage.corpo = TaskId
168.     } else {
169.         fmt.Printf(colorCode+"%2d: TaskId na eleição com mais prioridade que eu\n\033[0m", TaskId)
170.     }
171.     out <- inboundMessage
172. }
173. }
174. case TIPO_MSG_CONFIRM_ELECTION:
175. {
176.     alreadyVoted = false
177.     if !alreadyConfirmed {
178.         alreadyConfirmed = true
179.         fmt.Printf(colorCode+"%2d: confirmando resultado da eleição, processo líder é %2d\n\033[0m",
TaskId, inboundMessage.corpo)
180.         actualLeader = inboundMessage.corpo
181.         out <- inboundMessage
182.     } else {
183.         controle <- TaskId
184.     }
185. }
186. case TIPO_MSG_FINISH_PROCESS:
187. {
188.     fmt.Printf(colorCode+"%2d: finalizando o processo\n\033[0m", TaskId)
189.     keepListening = false
190. }
191. default:
192. {
193.     fmt.Printf(colorCode+"%2d: não conheço este tipo de mensagem\n\033[0m", TaskId)
194.     fmt.Printf(colorCode+"%2d: líder atual %d\n\033[0m", TaskId, actualLeader)
195.     keepListening = false
196. }
197. }
198. }
199.
200.     fmt.Printf(colorCode+"%2d: terminei \n\033[0m", TaskId)
201. }
202.
203. func main() {
204.
205.     wg.Add(5) // Add a count of four, one for each goroutine
206.
207.     // criar os processo do anel de eleicao
208.
209.     go ElectionStage(0, chans[3], chans[0], 0, "\033[31m") // este é o líder
210.     go ElectionStage(1, chans[0], chans[1], 0, "\033[34m") // não é líder, é o processo 0
211.     go ElectionStage(2, chans[1], chans[2], 0, "\033[33m") // não é líder, é o processo 0
212.     go ElectionStage(3, chans[2], chans[3], 0, "\033[32m") // não é líder, é o processo 0
213.
214.     fmt.Println("\n Anel de processos criado")
215.
216.     // criar o processo controlador
217.
218.     go ElectionControler(controle)
219.
220.     fmt.Println("\n Processo controlador criado\n")
221.     wg.Wait() // Wait for the goroutines to finish\
222. }

```